TREMOR at Scale — Project Plan

Oct 13, 2026 · @mokshad

Overview

TREMOR at Scale rebuilds TREMOR's Python ingestion as a crash-safe, privacy-first security log pipeline in Go, in about 7 weeks.

Pitch: ingests OpenTelemetry logs at 100k+ events/sec, masks PII before anything touches disk, detects anomalies and attacks in real time, and writes plain-English incident summaries with an LLM that explains but never decides.

Goals

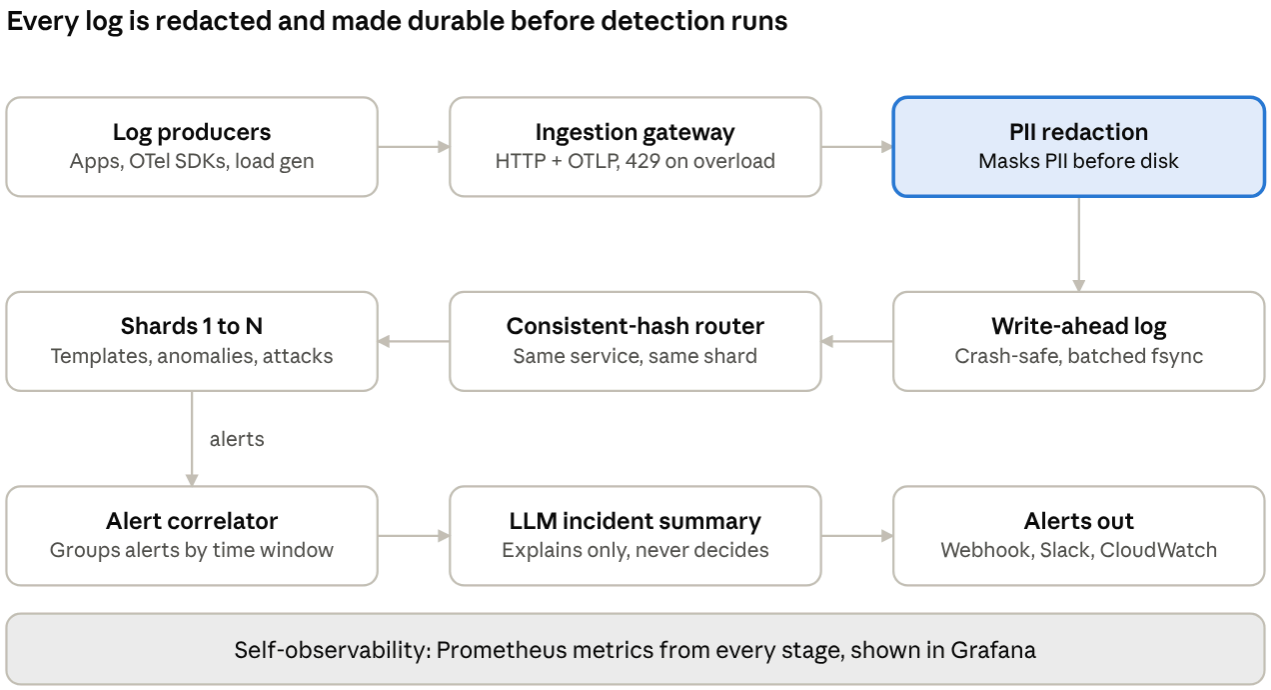
- Prove reliability: zero acknowledged events lost across 50 forced crashes.
- Prove throughput: 100k+ events/sec on a 4-core machine, benchmarked against Vector and Fluent Bit.
- Add a security layer that matches the cybersecurity specialization: brute force, credential stuffing, privilege changes, unusual access.
- Ship it publicly: open-source repo, demo video, blog post, real users.

Target resume line (fill in measured numbers):

> Built a crash-safe security log pipeline in Go: X events/sec on 4 cores, p99 ingest-to-alert under Y ms, zero loss across 50 crash tests, real-time brute-force detection, DPDPA-aligned PII redaction, and LLM incident summaries.

TREMOR was a 4-person team project (Acentra Health Build to Care). This is an individual extension in your own fork, and the resume should say so.

Architecture



TREMOR at Scale pipeline · 9 stages

Redaction and the write-ahead log come before any detection, so raw PII never reaches disk, Grafana or the LLM, and a crash never loses an acknowledged event.

Core engine

Four pieces carry the reliability and throughput claims. Build them first and never cut them.

1. Backpressure

- Bounded Go channels between every stage; nothing buffers without a limit.
- When the gateway's queue is full, return HTTP 429 (OTLP clients retry with backoff) or stop reading the TCP socket.
- Test: push 2x max throughput for 10 minutes. Memory stays flat, and every rejected event was told it was rejected.

2. Write-ahead log (crash recovery)

- Append each redacted event to segment files (e.g. 64 MB each) with a sequence number and checksum.
- Batch fsync (every N events or T ms) and acknowledge the producer only after its batch is synced.

- Each shard checkpoints the last sequence it processed; on restart, replay from the checkpoint.
- Delivery guarantee: at-least-once, with duplicates dropped by sequence number in the alert stage.
- Test: a chaos runner kill -9s the process at random points 50 times under load and checks every acknowledged event was processed.

3. Sharding

- Route by hash of `service.name` with consistent hashing, so one service's baseline state lives on one shard and needs no locks.
- One goroutine per shard; shard count defaults to CPU cores.
- Stretch: add or remove a shard and hand over its baseline state without losing it.

4. Drain port

- Port the Drain log-template algorithm (fixed-depth parse tree) from drain3 to Go.
- Validate against drain3 on the same log files: same templates, much faster.
- Carry over TREMOR's adaptive baselines and confirm they still hit 0 false alarms/hour with 100% recall on the original 10 labelled scenarios.

Features

Five features turn a fast pipeline into a security product, and each one ends in a measurable result.

1. OTLP ingestion

- Accept OpenTelemetry logs on the standard endpoints: OTLP/HTTP on port 4318 (`/v1/logs`, protobuf and JSON) and OTLP/gRPC on 4317.
- Map `service.name` from resource attributes to the shard key.
- Proof: point an OpenTelemetry Collector or an app's OTel SDK at TREMOR with a config change only.

2. PII redaction

- Runs right after the gateway and before the WAL, so raw PII never reaches disk, Grafana or the LLM.
- Detects emails, Indian phone numbers, Aadhaar numbers (12 digits + Verhoeff checksum to cut false positives), PAN, card numbers (Luhn check), and API keys/tokens (known prefixes + entropy).

- IPs and usernames are tokenized with a keyed hash (HMAC) instead of deleted. The same IP always maps to the same token, so attack detection still works without storing the raw value.
- Proof: precision and recall on a labelled synthetic PII set, plus the throughput cost of the stage.

3. Security detection

Rules run per shard on Drain templates and sliding-window counters, each mapped to MITRE ATT&CK.

Detection	Signal	ATT&CK
Brute force	Many failed logins for one user from one source in a short window	T1110.001
Credential stuffing	Many distinct usernames failing from one source, or low-and-slow across many sources	T1110.004
Privilege change	Role grants, sudo or admin changes outside the service's baseline	T1098, T1548
Unusual access	Successful login from a new source or at an unusual hour for that user	T1078

- Test data: an attack simulator built into the load generator, plus public SSH logs from Loghub.
- Proof: detection rate on simulated attacks and false alarms per hour on normal traffic.

4. Grafana dashboard

- Expose Prometheus metrics: events/sec in and out, 429 rate, queue depth per shard, WAL lag and fsync latency, redactions by type, alerts by type, LLM latency and cost.
- Ship a ready-made Grafana dashboard JSON so `docker compose up` shows it immediately.
- Proof: the demo video shows the dashboard live through a load test and a crash.

5. LLM incident summaries

- The alert correlator groups alerts within a time window across services into one incident.

- The summarizer sends the redacted incident context (alerts, new templates, counts, recent deploy events) to an LLM and gets structured JSON back: summary, likely trigger, affected services, suggested next check, and the alert IDs it relied on.
- Detection stays deterministic. The LLM only explains; it never creates, suppresses or closes an alert. Same principle as StockGuard.
- Reuse StockGuard's provider-agnostic LLM layer (Groq by default). Cache per incident, rate-limit, and fall back to a template summary if the LLM is down.
- Proof: on the 10 TREMOR scenarios plus the attack scenarios, score whether each summary names the right service and trigger.

Tech stack and repo structure

Go runs the whole pipeline; Python appears only in evaluation scripts.

Area	Choice
Language	Go (goroutines and channels fit backpressure and sharding)
Ingestion	net/http, gRPC, OpenTelemetry protobuf definitions
Storage	Custom WAL segment files on local disk
Metrics	Prometheus client_golang + Grafana
LLM	Groq through StockGuard's provider layer
Profiling	pprof, Go benchmarks
Packaging	Docker, Docker Compose, GitHub Actions CI

```
tremor/
  cmd/tremor/          server
  cmd/loadgen/         load + attack simulator
  cmd/chaos/           kill -9 crash test runner
  internal/ingest/     HTTP + OTLP receivers
  internal/redact/     PII detection and tokenizing
  internal/wal/        segments, fsync, checkpoints
  internal/router/     consistent hashing
  internal/drain/      template mining
  internal/detect/     anomaly + security rules
  internal/summarize/  correlator + LLM
  internal/metrics/    Prometheus metrics
  deploy/              docker-compose, Grafana dashboard
  bench/               scripts + results
  docs/                architecture, blog draft
```

Build plan

Seven weeks: weeks 1–3 build the engine, 4–6 add the features, week 7 proves and launches it. This assumes roughly 15–20 hours a week alongside classes; if a week slips, use the cut order at the end.

Week 1: Gateway and backpressure

- [ ] Go HTTP gateway with bounded channels and 429 on overload
- [ ] Load generator with configurable rate and number of services
- [ ] First throughput number recorded in `bench/`

Week 2: Durability

- [ ] WAL segments with checksums and batched fsync
- [ ] Per-shard checkpoints and replay on restart
- [ ] Chaos runner: 50 kill -9 crashes, zero acknowledged events lost

Week 3: Sharding and detection core

- [ ] Consistent-hash router, one goroutine per shard
- [ ] Drain ported to Go and matched against drain3
- [ ] Adaptive baselines pass TREMOR's 10 labelled scenarios

Week 4: OTLP and PII redaction

- [ ] OTLP/HTTP and OTLP/gRPC receivers, tested with an OTel Collector
- [ ] Redaction stage before the WAL, HMAC tokenizing for IPs and usernames
- [ ] Redaction precision/recall and throughput cost measured

Week 5: Security detection

- [ ] Brute force, credential stuffing, privilege change and unusual access rules
- [ ] Attack simulator in loadgen; Loghub SSH logs replayed
- [ ] Detection rate and false alarms per hour recorded

Week 6: Grafana and LLM summaries

- [ ] Prometheus metrics and Grafana dashboard in Docker Compose
- [ ] Alert correlator and LLM summarizer with template fallback
- [ ] Summary accuracy scored on all scenarios

Week 7: Benchmark and launch

- [ ] Vector and Fluent Bit comparison on the same machine
- [ ] pprof pass with one documented optimization
- [ ] README, demo video and blog post

Testing and benchmarks

These are targets, not results: replace each with the measured number before it goes on the resume.

Claim	Target	How it's measured
Throughput	100k+ events/sec on 4 cores (stretch: 150k)	Raise loadgen rate until 429s start
Latency	p99 ingest-to-alert under 500 ms	Gateway timestamp vs. alert emit time
Durability	0 acknowledged events lost in 50 crashes	Chaos runner compares acked vs. processed sequence numbers
Overload	Flat memory at 2x max load for 10 min	Prometheus memory + 429 counts

Claim	Target	How it's measured
Anomaly detection	0 false alarms/hr, 100% recall on error-rate anomalies	TREMOR's 10 labelled scenarios
Attack detection	Every simulated attack caught; false alarms/hr reported	Attack simulator + Loghub SSH logs
PII redaction	Precision/recall reported; under 15% throughput cost	Labelled PII set; run with and without the stage
LLM summaries	Right service and trigger in at least 8 of 10 incidents	Manual rubric per incident

Comparison with Vector and Fluent Bit

- Run them on the same machine, same input files and rates, doing comparable parse-and-forward work.
- You don't need to beat them. Report honestly and explain the trade-off: TREMOR also runs redaction and detection.
- Publish machine specs and benchmark scripts so anyone can reproduce the numbers.

Launch plan

Launching is what turns the build into interviews, so it gets its own steps.

1. **README:** one-line pitch, architecture diagram, benchmark table, one-command quick start.
2. **Demo video (2–3 min):** a load test on Grafana, kill -9 mid-run, recovery with zero loss, then a brute-force attack caught with its LLM summary.
3. **Blog post:** what broke and how you fixed it, e.g. "Surviving 50 kill -9 crashes with zero data loss."
4. **Share:** LinkedIn post with the video, r/golang, r/devops, Show HN, Gophers Slack, a short talk at a Chennai Go or DevOps meetup.
5. **Real users:** offer it to icebrkr, college clubs running servers, and small startups; label good-first-issues on GitHub.
6. **Resume:** replace the TREMOR entry with measured numbers and the demo link.
7. **Outreach:** send the demo to engineers on infra and security teams at target companies and ask for feedback.

Interview prep and scope cuts

Write your answers to these before any interview; interviewers dig hardest into your strongest project.

- Why bounded channels, and what does a producer do when it gets a 429?
- What is your delivery guarantee, and how do you handle duplicates?
- What fsync batching interval did you choose, and what can a crash lose before an ack?
- Why redact before the WAL, and how does tokenizing keep attack detection working?
- How do you stop credential-stuffing rules from flagging a busy office NAT gateway?
- Why can the LLM never decide, and what happens when it's down or wrong?
- How would you scale this to 10 machines?

Risks

Risk	Fallback
Scope creep pushes past 7 weeks	Follow the cut order below
Vector or Fluent Bit are much faster	Report honestly and explain the redaction and detection cost
LLM cost or rate limits	Cache per incident, rate-limit, template fallback
PII rules mangle normal log text	Checksums for Aadhaar and cards; report precision

Cut order if behind (cut from the top first):

1. OTLP/gRPC (keep OTLP/HTTP)
2. Shard rebalancing stretch goal
3. Unusual-access rule
4. Grafana polish (keep the metrics)

Never cut the WAL, crash tests, benchmarks or PII redaction: they carry the headline claims.